

TEST AUTOMATION



Contents

1

- Introduction

2

- CANoe + CAPL

3

- vTESTstudio

4

- ECU-TEST



Definition

Automation Testing is the process of using tools, scripts, and software to perform test cases by repeating pre-defined actions. Test Automation focuses on replacing manual human activity with systems or devices.



Validation
Workflow



Why Automation Testing

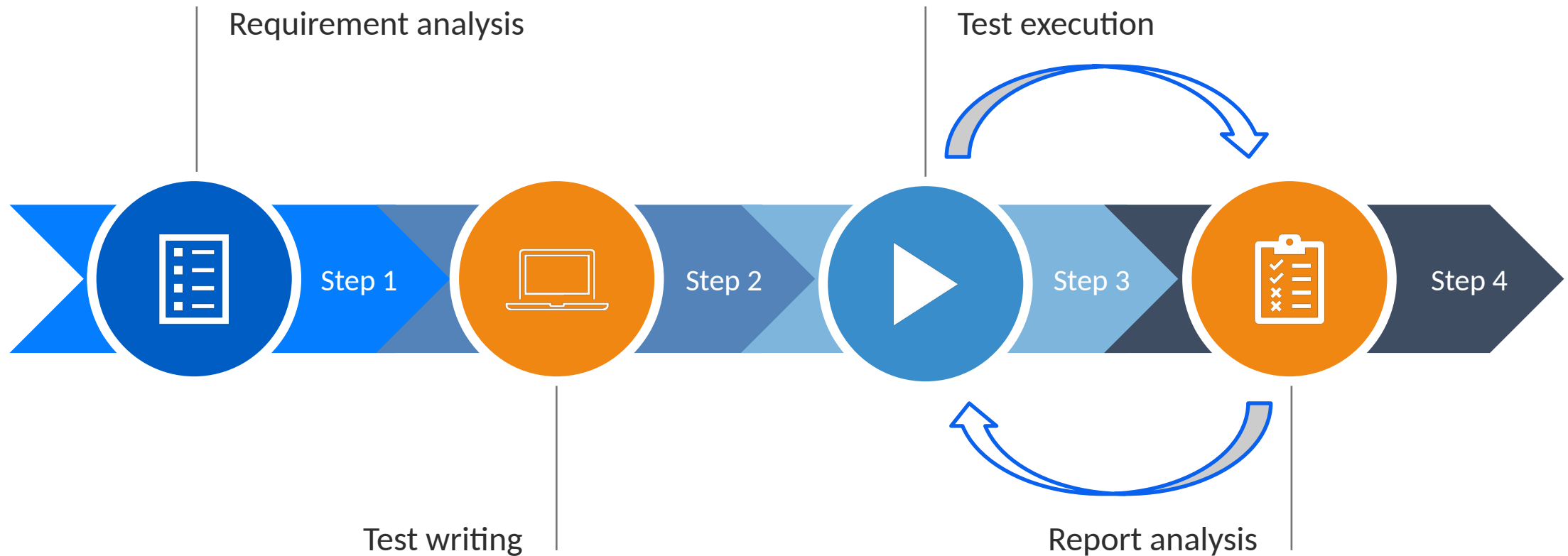


Pros & Cons



Automation Tools

Manual Validation Workflow

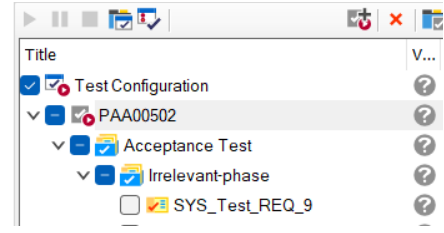


k Auto Validation Workflow

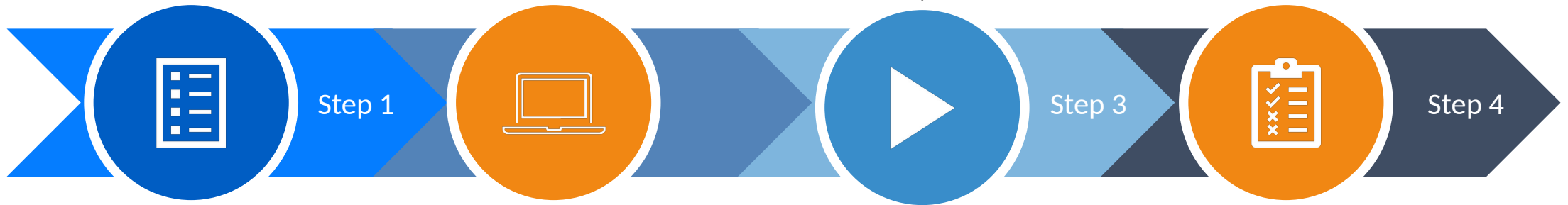
SYS - Requirements

As soon as receive the first Power supplied, the MCU shall configured the SBC Wake-up reason from ID 0x521 and during the configuration shall not communicate by H-CAN

Requirement analysis



Test execution



Test writing

SYS - TC Step Description	SYS - TC Expected Results
Step 1: Wake up the OBC;	Step 1: Verify: - OBC wakes up; - GhdcpExposed1CanNmFrame (ID 0x521) is sent on H-CAN; - OBC communicates on H-CAN;
Step 2: Set OBC in Sleep;	Step 2: Verify that OBC goes in Sleep;

Test automation

Test Case	SYS_Test_REQ_9_Neg
Preparation	Preparation
StepDescription	Step 1
Set	DeactivateNM=1 Set StateControl::DeactivateNM=1
Wait	500 ms Wait for 500 ms

Report analysis

CAPL Inline: None		
835949	Step 3	Wait for 20 s; -> Verify that OBC is in Charge;
Wait: None		
835949	1	Waited for 20.000 s
Check: Passed		
835949	1	[+] Check of expected values
Symbol		
System variable 'Rte_Irv_AC_St		
CAN signal 'CMD_OBC_Control		
PDU signal 'isOnBdChgrSts3'		



Why Automation Testing?



Reliable

Automation testing improves accuracy and reduces human-generated error.



Fast

Automation increases the speed of test execution. It can perform testing much faster than a human can.



Repeatable

The same tests are performed quickly whenever the new software is released.



Versatile

Automation allows most of the tests already written to be used in new projects as well.



Flexible

Automation allows testing even at night.



Efficient

It's possible to execute a huge number of tests in just 24 hours.



Pros & Cons

Advantages

- Overall costs
- Dangerous tests
- Hard conditions
- Auto-reports

Limitations

- Not applicable to all tests
- Initial investment



Automation levels



Level 1

Only a part of tests automated.



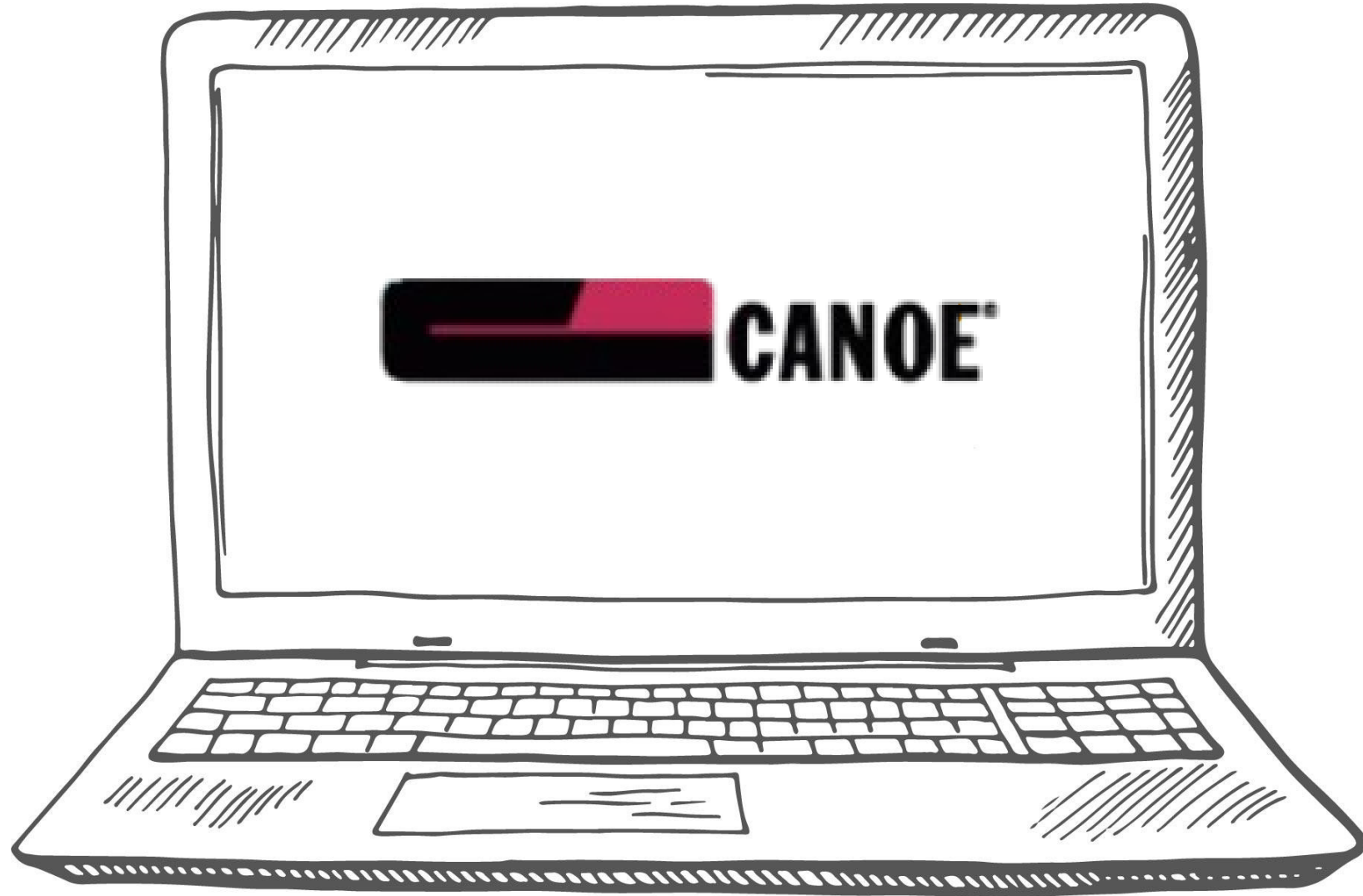
Level 2

All tests automated.
All the other validation steps manually executed.



Level 3

All tests automated.
Also other validation steps automated.



k CANoe - Simulation

- o Vector tool for validation activities
- o Simulation networks from database
- o Signal logic not simulated
- o Signal values editable

The screenshot displays the Vector CANoe Simulation environment. The top menu bar includes File, Home, Analysis, Simulation, Test, Diagnostics, Environment, Hardware, Tools, and Layout. Below the menu is a toolbar with icons for Simulation Setup, Model Generation Wizard, Node/Network Panels, Security Configuration, Interactive Generator, LIN ISC, LIN NM, Automation, and Signal Generators. The main workspace shows a network diagram with 13 ECUs (AMP, BCM, DCSD, DIAG_TEST, DTV, ECC, EMCM, ETM, HCPM, IPC, LSMD, LSMP, LTM) connected to a CAN bus. The bottom panel shows the 'ECC | Messages' window with a table of signals and their values.

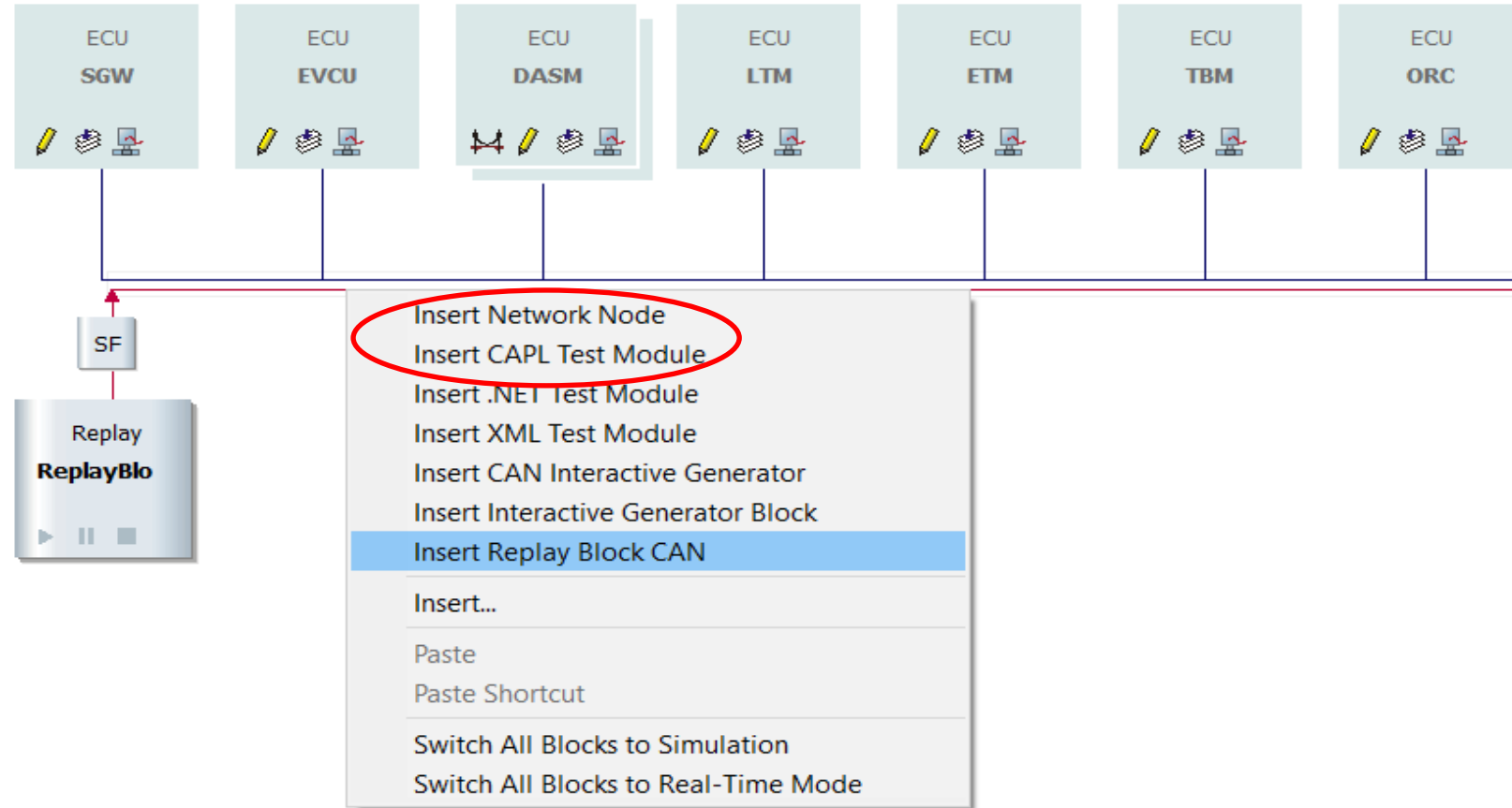
Signal	Physical ...	Raw Value	Symbolic Value
CompressorACReqSts	0	0x0	Not_Active
FHeatedWindowCmd	0	0x0	Not_Active
HVAC_Blower_Speed	0	0x0	-
MaxDefrostSts	0	0x0	Not_Active
PowerModeSts_Climate	0	0x0	Active
RHeatedWindowCmd	0	0x0	Standard_Power
StopStartClimateInfo	0	0x0	Not_Active
StopStartEnable	0	0x0	SPST_ON
StopStartEnable	0	0x0	False

The right sidebar shows the 'Networks' tree with a hierarchy: CAN Networks > CAN > Nodes > AMP, BCM, DCSD, DIAG_TEST, DTV, ECC, EMCM, ETM, HCPM, IPC, LSMD, LSMP, LTM, TBM. Below this are 'Generators', 'Interactive Generators', 'Replay blocks', 'Databases', 'CAN_BH_Vehicle', 'Channels', 'CAN2', and 'LIN Networks'.

k CANoe – CAPL scripts

0 Network Nodes

0 CAPL Test Modules



k CANoe – Network node

- 0 ECU software simulation
- 0 Panels control
- 0 HIL control
- 0 Networks simulation
- 0 ...

```
87 //Declare a variable that activate all instrumentation measurement only if is in set ON
88 on sysvar sysvar::Aux::StartSimulation
89 {
90     if(@sysvar::Aux::StartSimulation == 1)
91     {
92         // Start VT7001 and set LVDC power supply command
93         sysvar::VTS::Power_SupInt.SetRefVoltageMode(1);
94         @sysvar::VTS::Power_SupInt::RefVoltage = 3.0;
95         sysvar::VTS::Power_Out1.ResetMinMax();
96         sysvar::VTS::Power_Out2.ResetMinMax();
97         @sysvar::VTS::Power_Sup2::RefVoltage = 0;
98         sysvar::VTS::Power_Sup2.SetRefVoltageMode(0);
99         @sysvar::VT7001::Started = 1;
100         @sysvar::VTS::Power_SupInt::RefVoltage = @sysvar::VT7001::Ch2_Voltage;
101         SerialPowerSupplyVoltage(@sysvar::VT7001::Ch1_Voltage);
102         SerialPowerSupplyCurrent(@sysvar::OBC::KL30::CURR_Limit);
103         @sysvar::OBC::KL30::SW = 0;
104         @sysvar::VTS::Power_Out2::Active = 1;
105         // Start the timer
106         setTimerCyclic(tTimerRefresh,100);
107     }
108     else if (@sysvar::Aux::StartSimulation == 0)
109     {
110         // Stop VT7001
111         sysvar::VTS::Power_SupInt.SetRefVoltageMode(0);
112         SerialPowerSupplySwitch(0);
113         @sysvar::OBC::KL30::SW = 0;
114         @sysvar::VTS::Power_Out1::Active = 0;
115     }
116 }

198 // Automatic CHARGE state
199 //
200 //-----
201 //*****
202 //
203 // Switch in Charge state when the button is pressed
204 on sysvar sysvar::Aux::Manual_Button::Charge
205 {
206     if(@this == 1){
207         // charge_request = 1;
208         // setTimer(Timer_State_STANDBY, 200);
209
210         //AC Connection
211         @sysvar::OBC::Aux::PLUG = 1;
212
213         //GcccExposed1CanSignalIPdu05
214         $VectorSimulationNode::GcccExposed1CanSignalIPdu05::isOnBdChrgrEna = 1;
215         $VectorSimulationNode::GcccExposed1CanSignalIPdu05::isOnBdChrgrEna_UB = 1;
216         $VectorSimulationNode::GcccExposed1CanSignalIPdu05::isOnBdChrgrHvIDcReq = 500;
217         $VectorSimulationNode::GcccExposed1CanSignalIPdu05::isOnBdChrgrHvIDcReq_UB = 1;
218         $VectorSimulationNode::GcccExposed1CanSignalIPdu05::isOnBdChrgrHvULimMax1 = 16000;
219         $VectorSimulationNode::GcccExposed1CanSignalIPdu05::isOnBdChrgrHvULimMax1_UB = 1;
220         $VectorSimulationNode::GcccExposed1CanSignalIPdu05::isOnBdChrgrHvULimMin = 0;
221         $VectorSimulationNode::GcccExposed1CanSignalIPdu05::isOnBdChrgrHvULimMin_UB = 1;
222
223         //HvbmSignalIPdu03_100
224         $VectorSimulationNode::HvbmSignalIPdu03_100::isHvBattChrgrInhbnSafeStopChrgrn = 0;
225         $VectorSimulationNode::HvbmSignalIPdu03_100::igHvBattChrgrInhbnSafe_UB = 1;
226
227         //GCCCEXPOSED1CANPdu0
228         $VectorSimulationNode::GCCCEXPOSED1CANPdu0::isOnBdChrgrModReq = 1;
229     }
```

k CANoe – CAPL Test Module

```
3 includes
4 {
5     #include "SimDiagECU.can"
6 }
7
8 variables
9 {
10     diagReq;
11 }
12
13 void MainTest()
14 {
15     write(" ");
16     write("Diagnosis");
17     write("-----");
18     write("Testcase1_Read");
19     Testcase1_Read();
20     write("Testcase2_Write");
21     Testcase2_Write();
22     write("Again");
23     Testcase1_Read();
24     write("Testcase3_Ser");
25     Testcase3_Ser();
26 }
27
28 testcase Testcase1_Read()
29 {
30     DiagRequest Door.SerialNumber_Read req;
31     DiagResponse Door.SerialNumber_Read resp;
32     char buffer[100];
33     long ret, ret2, ret3, ret4;
34     byte serialNumber[4];
35
36     testStep(0, "1.0", "Read serial number");
37
38     diagGetObject(req, buffer, elCount(buffer));
39     DiagSendRequest(req);
40     ret = testWaitForDiagRequestSent(req, 2000);
41
42     if(ret==1) // Request sent
43     {
44         TestReportWriteDiagObject(req);
45         write("Request has been successfully sent");
46         ret2=TestWaitForDiagResponse(req, 5000);
47
48         if(ret2==1) // Response received
49         {
50             ret3=DiagGetLastResponse(req);
51             if(ret3>=0)
52             {
53                 if(diagIsPositiveResponse(req))
54                 {
55                     DiagGetParameterRaw(req, serialNumber);
56                     write("Current serial number: ");
57                     TestReportWriteDiagObject(req);
58                     testStepPass(0, "1.0");
59                 }
60             }
61         }
62     }
63
64     if(ret<0)
65     {
66         write("Internal error %d occurred while trying to send request %s", ret, buffer);
67         testStepFail(0, "1.0", "Internal error %d occurred while trying to send request %s", ret, buffer);
68     }
69 }
70
71 testcase Testcase2_Write()
72 {
73     DiagRequest Door.SerialNumber_Write req;
74     char buffer[100];
75
76     long ret, ret2, ret3;
77     byte serialNumber[4] = {0x44, 0x55, 0x66, 0x77};
78
79     testStep(0, "2.0", "Write serial number"); // Write the current teststep to the testreport
80
81     diagGetObject(req, buffer, elCount(buffer)); // Get the name of the diagnostic object
82     // Write the serial number to the request object in raw format
83     diagSetParameterRaw(req, "SerialNumber", serialNumber, elCount(serialNumber));
84
85     DiagSendRequest(req);
86     ret = testWaitForDiagRequestSent(req, 2000); // Wait until the request has been completely sent
87
88     if(ret==1) // Request sent
89     {
90         TestReportWriteDiagObject(req); // Write the request object to the testreport
91         write("Request has been successfully sent");
92         ret2=TestWaitForDiagResponse(req, 5000); // Wait for a response, here for 5000ms. Note: This is no P2 timeout!
93     }
94 }
```




CANoe – Test report

4 PreChargingTest2: Passed

Test case begin: 2015-01-13 16:41:37 (logging timestamp 39.329111)

Test case end: 2015-01-13 16:41:42 (logging timestamp 44.942227)

Main Part of Test Case

Timestamp	Test Step	Description	Result
44.541412	Resume reason	Resumed on signal 'Signal:CAN1/PowerTrain/MSG631/CR_Qc_ChargerMod' Elapsed time=5212.3ms (max=10000ms)	-
44.541412	1	Get Mode - CC	pass
44.942227	Resume reason	Resumed on signal 'Signal:CAN1/PowerTrain/MSG631/CR_Qc_ChargerMod' Elapsed time=400.815ms (max=10000ms)	-
44.942227	1	Get CV mode	pass

5 ChargingPeriod: Passed

Test case begin: 2015-01-13 16:41:42 (logging timestamp 44.942227)

Test case end: 2015-01-13 16:41:53 (logging timestamp 55.942227)

Main Part of Test Case

Timestamp	Test Step	Description	Result
45.942227	Resume reason	Elapsed time=1000ms (max=1000ms)	-
45.942227	1		pass
55.942227	Resume reason	Elapsed time=10000ms (max=10000ms)	-
55.942227	1		pass

6 StopChargingTest: Passed

Test case begin: 2015-01-13 16:41:53 (logging timestamp 55.942227)

Test case end: 2015-01-13 16:41:54 (logging timestamp 56.064674)

Main Part of Test Case

Timestamp	Test Step	Description	Result
56.064674	Resume reason	Resumed on signal 'Signal:CAN1/PowerTrain/MSG631/CF_Qc_ChgFinished' Elapsed time=122.447ms (max=10000ms)	-
56.064674	1	Charging stopped	pass

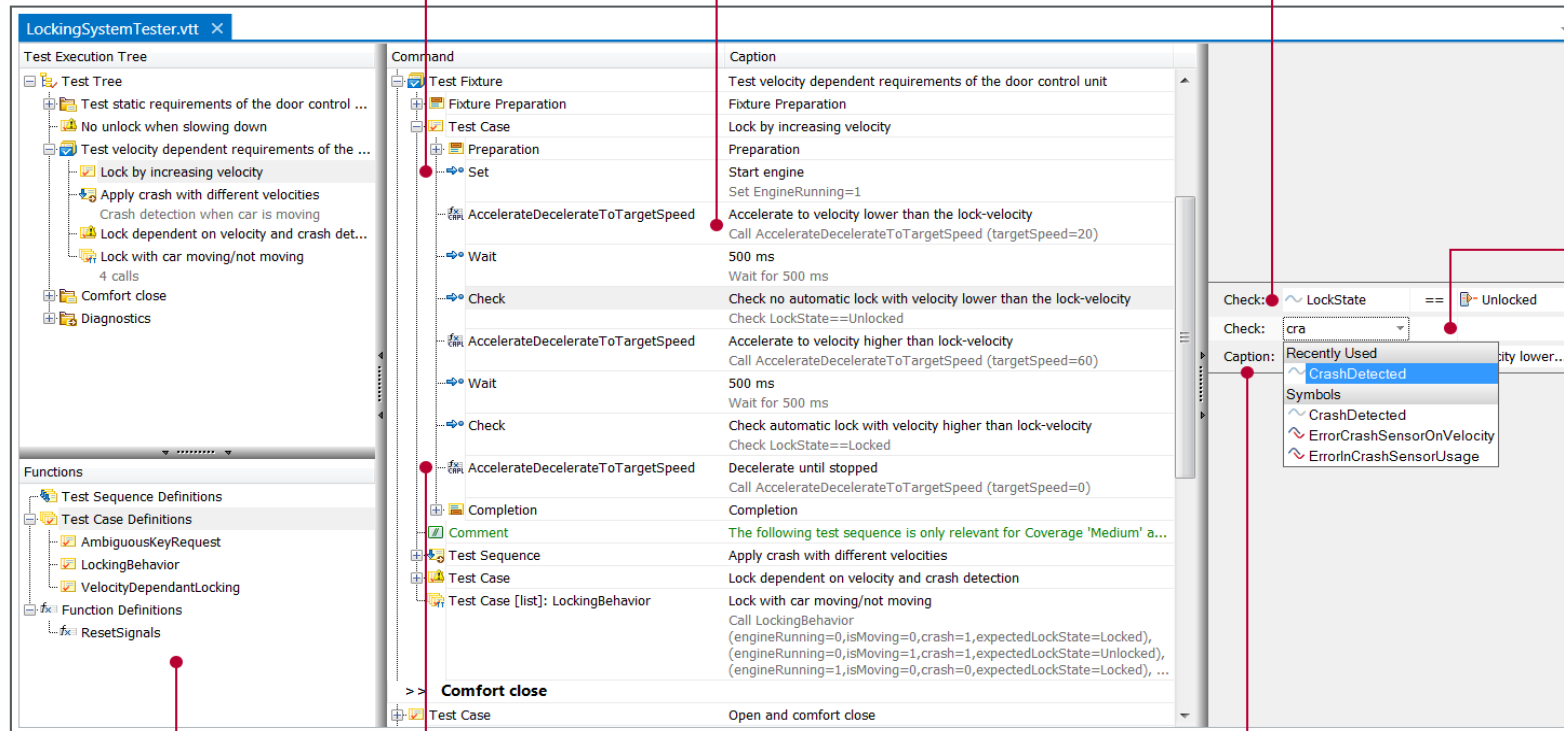


vTESTstudio – Test Table Editor

Easy definition of test sequences
by pre-defined test commands

Easy readability by summary of
command parametrization

Comfortable support of test step
parameterization by drag & drop



The screenshot displays the vTESTstudio Test Table Editor interface. On the left, a 'Test Execution Tree' shows a hierarchy of test cases, including 'Test static requirements of the door control ...', 'Test velocity dependent requirements of the ...', 'Lock by increasing velocity', 'Apply crash with different velocities', 'Lock dependent on velocity and crash det...', 'Lock with car moving/not moving', 'Comfort close', and 'Diagnostics'. Below this, a 'Functions' panel lists 'Test Sequence Definitions', 'Test Case Definitions', 'AmbiguousKeyRequest', 'LockingBehavior', 'VelocityDependantLocking', and 'Function Definitions' with a 'ResetSignals' function. The main area shows a 'Command' table with columns for 'Caption' and 'Command'. The 'Caption' column contains a summary of the test sequence, while the 'Command' column lists the specific commands and their parameters. The 'Command' table includes steps like 'Test Fixture', 'Fixture Preparation', 'Test Case', 'Preparation', 'Set', 'AccelerateDecelerateToTargetSpeed', 'Wait', 'Check', 'Decelerate until stopped', 'Completion', 'Comment', 'Test Sequence', 'Test Case', and 'Test Case [list]: LockingBehavior'. The 'Test Case [list]: LockingBehavior' step is expanded, showing a list of test cases with their parameters. On the right, a 'Check' table shows a check for 'LockState' with a value of 'Unlocked'. A dropdown menu is open, showing 'Recently Used' symbols: 'CrashDetected', 'CrashDetected', 'ErrorCrashSensorOnVelocity', and 'ErrorInCrashSensorUsage'. The 'CrashDetected' symbol is selected.

Definition of reusable
test cases and functions

Direct calls to CAPL and C#
test cases and functions possible

Autocomplete for symbols

Access to
value table
entries
defined in
databases







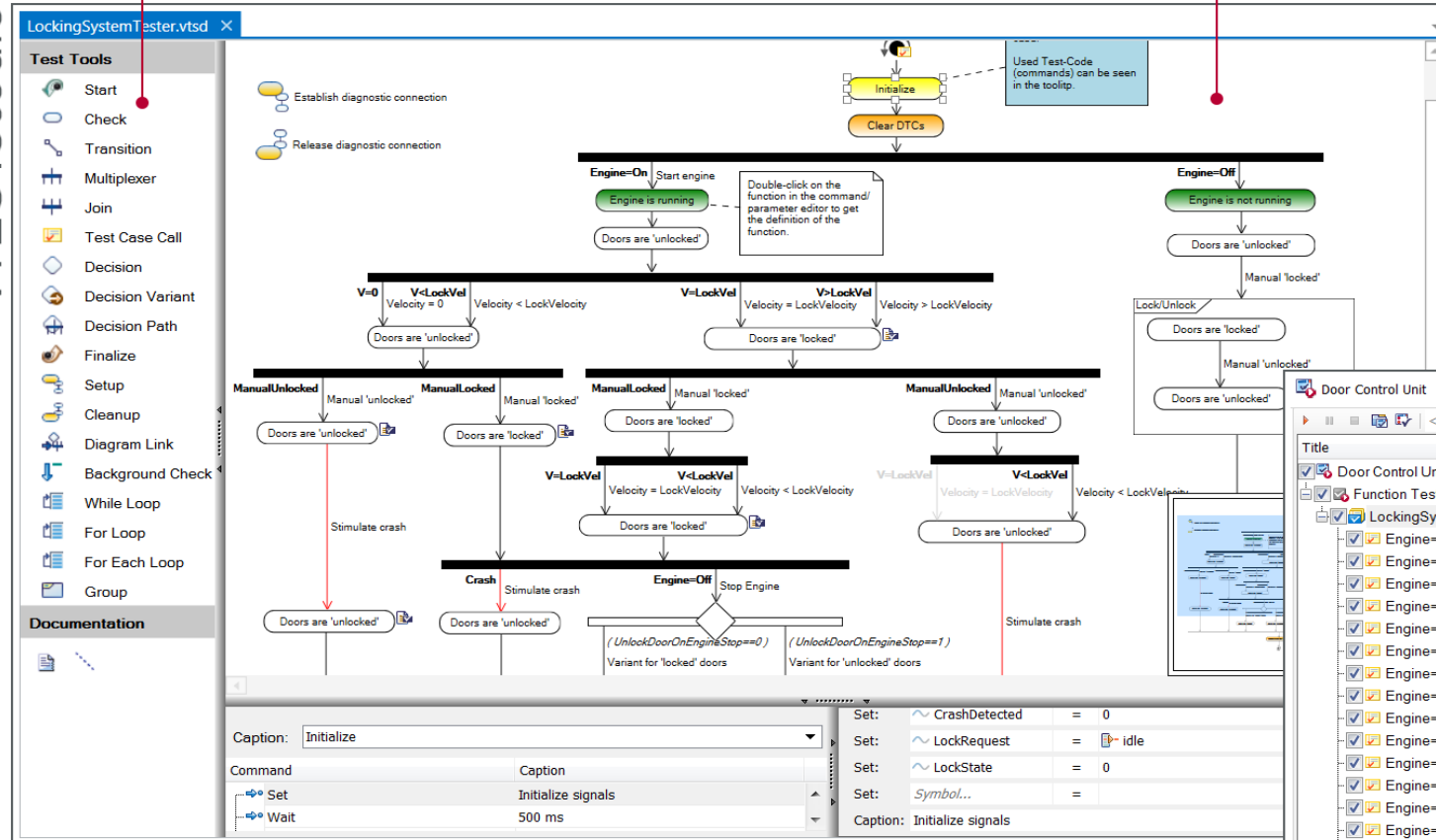
vTESTstudio – Test Sequence Diagram

Easy definition of diagrams by drag & drop of graphical elements out of a toolbox

Test sequence diagram for a clear and concise representation

Automatic generation of one test case for each path through the diagram

vTESTstudio



Door Control Unit

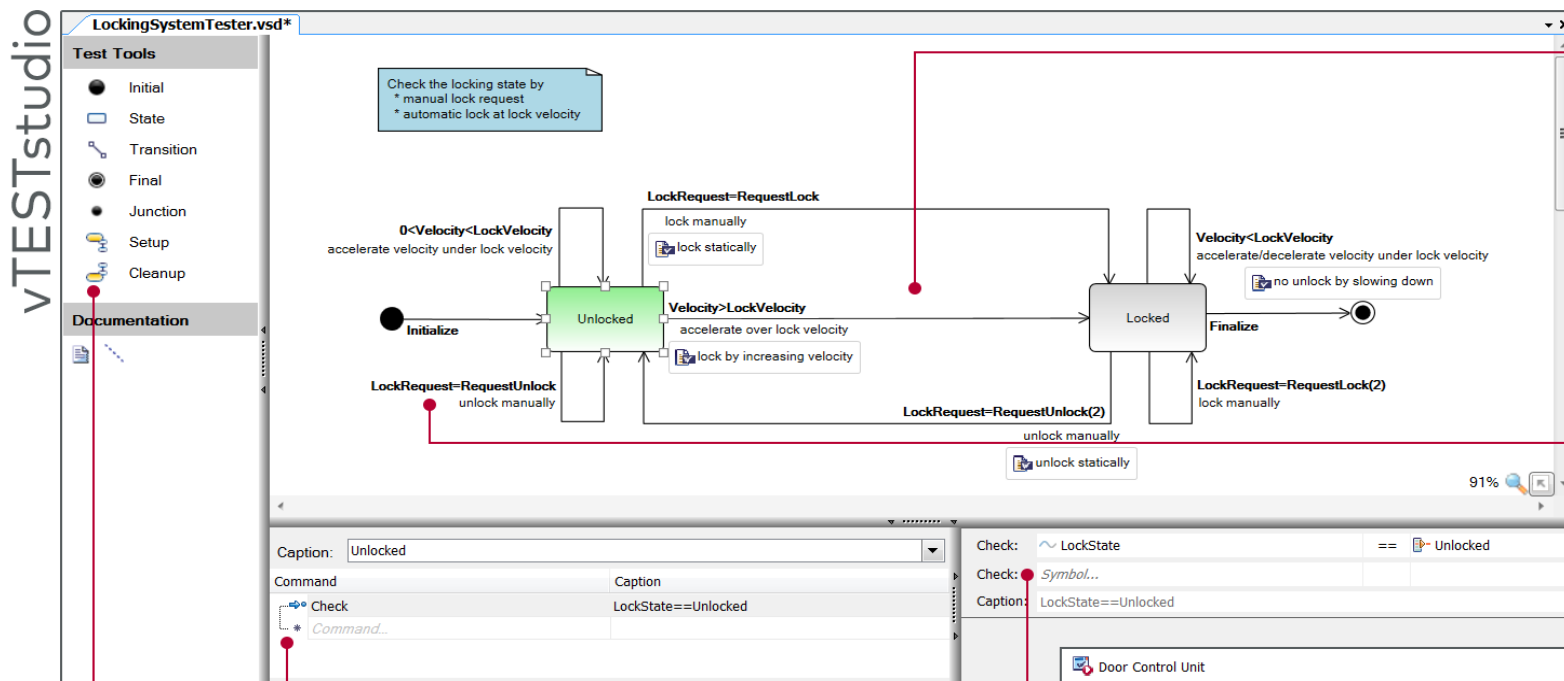
Title	Verdict	Runtime
Door Control Unit	?	?
Function Test	?	?
LockingSystemTester	?	?
Engine=On V=0 ManualUnlocked	?	?
Engine=On V=0 ManualLocked Crash	?	?
Engine=On V=0 ManualLocked Engine=Off	?	?
Engine=On V<LockVel ManualUnlocked	?	?
Engine=On V<LockVel ManualLocked Crash	?	?
Engine=On V<LockVel ManualLocked Engine=Off	?	?
Engine=On V=LockVel ManualLocked V=LockVel Crash	?	?
Engine=On V=LockVel ManualLocked V=LockVel Engine=Off	?	?
Engine=On V=LockVel ManualLocked V<LockVel Engine=Off	?	?
Engine=On V=LockVel ManualUnlocked V=LockVel	?	?
Engine=On V>LockVel ManualLocked V=LockVel Crash	?	?
Engine=On V>LockVel ManualLocked V=LockVel Engine=Off	?	?
Engine=On V>LockVel ManualLocked V<LockVel Crash	?	?
Engine=On V>LockVel ManualLocked V<LockVel Engine=Off	?	?
Engine=On V>LockVel ManualUnlocked V<LockVel	?	?
Engine=Off	?	?

00:00:00

CANoe



vTESTstudio – State Diagram



Expected behavior of the SUT modeled as state transition diagram

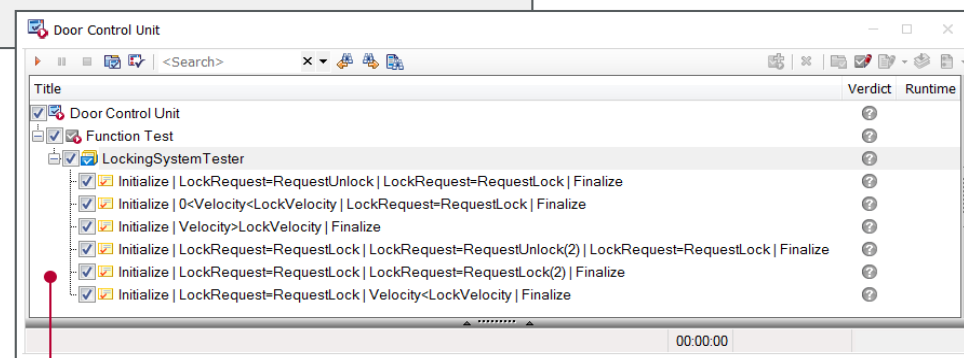
Extension of the system model to test model e.g. by (self-)transitions as unspecified scenarios

Access to symbols by drag & drop and autocomplete

Usage of pre-defined commands for:

- stimulation and checks
- user-defined functions, e.g. in CAPL C#

Definition of state diagrams by drag & drop of graphical elements out of a toolbox



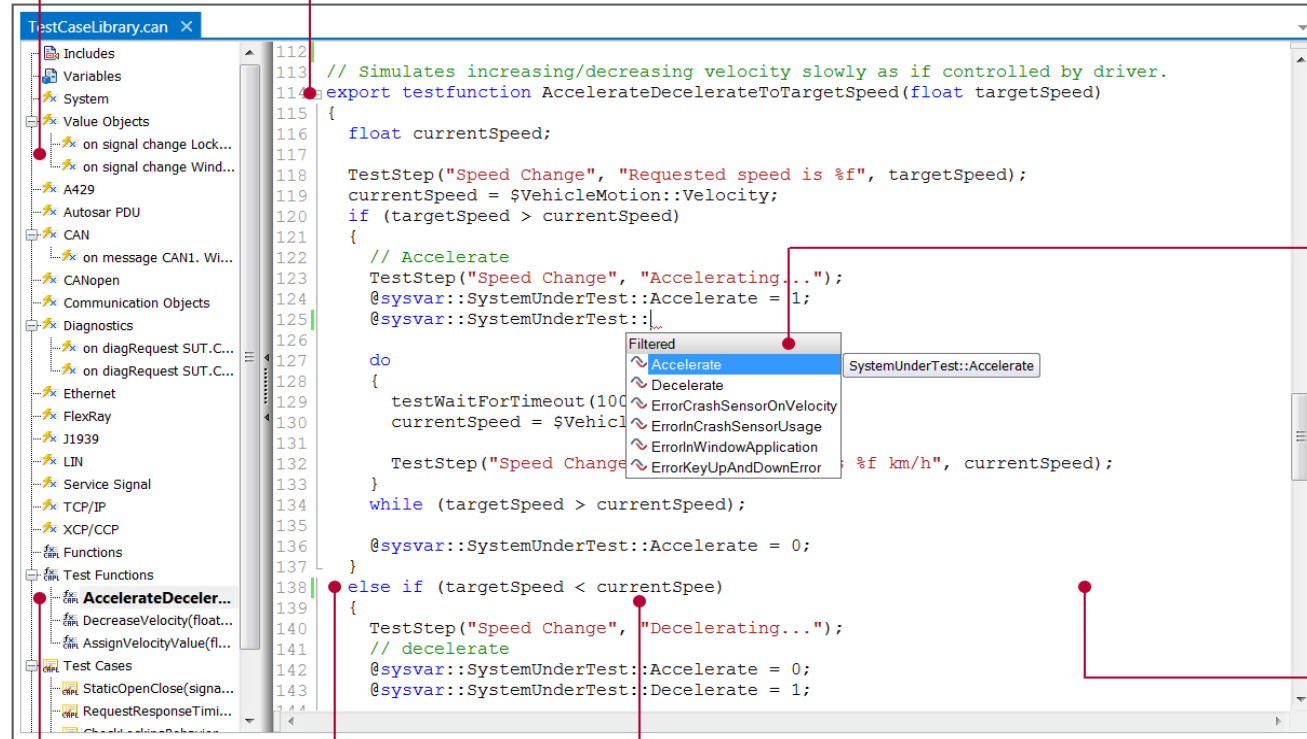
Automatic test case generation based on transition coverage; several generation algorithms supported



vTESTstudio – Programming Editor

Definition of event handlers for signal value pre-processing

Definition of interface functions for usage in other test design languages



Autocomplete with support of symbols as well

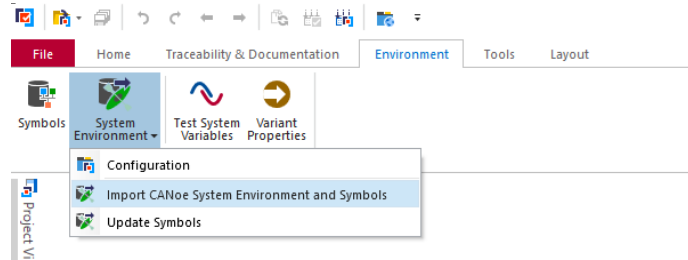
Test-specific API

Navigator for easy navigation within one file

Error underlining

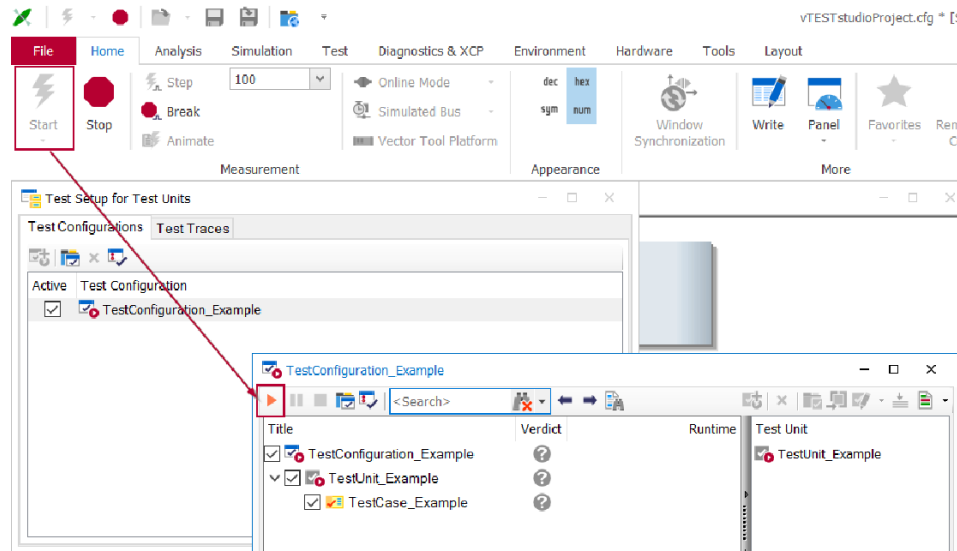
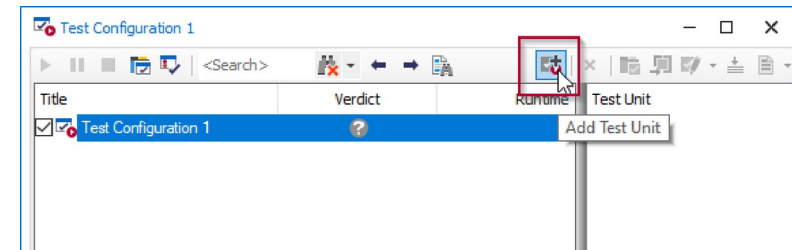
Syntax highlighting

vTESTstudio – CANoe



... add Test Unit in Test Configuration ...

Import CANoe configuration in vTESTstudio ...



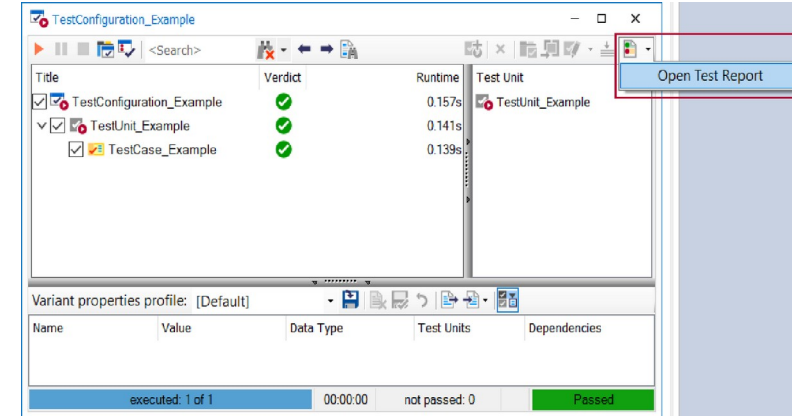
... select tests and run them during CANoe measurement.

vTESTstudio – Test report

Test begin: 2024-11-06 17:00:28 (logging timestamp 1.135036)
Test end: 2024-11-06 17:02:08 (logging timestamp 101.350000)

Statistics

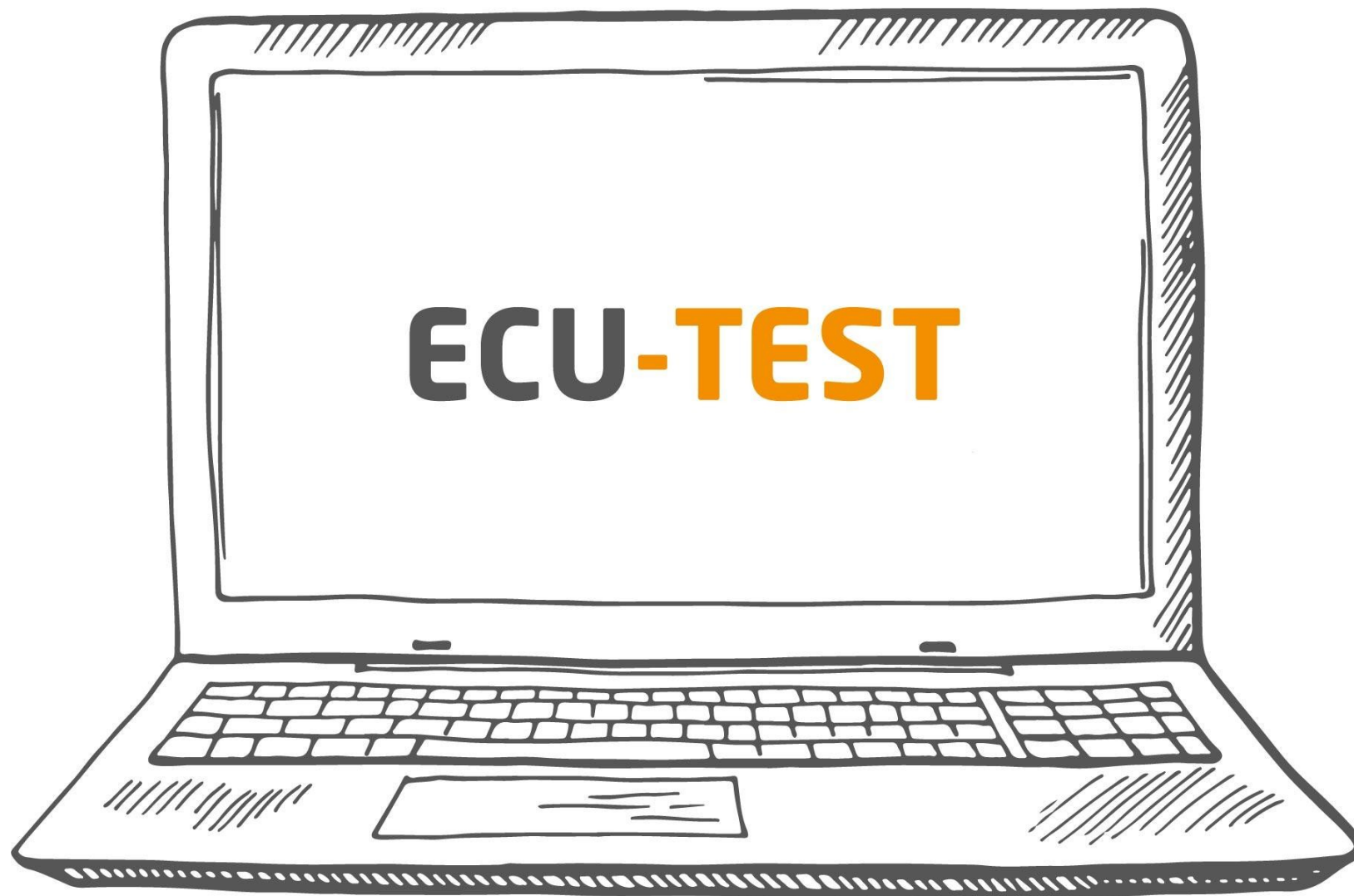
Executed test cases	1	
Test cases passed	1	100% of executed test cases
Test cases inconclusive	0	0% of executed test cases
Test cases failed	0	0% of executed test cases
Test cases with error in test system	0	0% of executed test cases
Number of failures outside of test cases		
Inconclusive	0	
Fail	0	
Error in test system	0	
Not executed test cases		
Some elements were not executed because they were deactivated by the tester.		



Test Results

1	PAA00502	pass
1.1	Acceptance Test	1
	Irrelevant-phase	not executed
	Single-phase	not executed
	NAR	not executed
	Three-phase	not executed
1.1.1	Charge Test	1
	3ph_charge	not executed
	3ph_charging_ramp	not executed
1.1.1.1	1ph_charge	pass
	1ph_charging_ramp	not executed
	Other	not executed

20. CAPL Inline: None						
39.835949	Step 3	Wait for 20 s; -> Verify that OBC is in Charge;				
21. Wait: None						
59.835949	1	Waited for 20.000 s				
22. Check: Passed						
59.835949	1	[-] Check of expected values				
		Symbol	Op.	Reference value	Actual	Res
		System variable 'Rte_Irv_AC_StateManager_IRV_AC_CurrentState'	==	4 (CHARGE_CC)	4	pas
		CAN signal 'CMD_OBC_Control_Action'	==	2 (Charge)	2	pas
		PDU signal 'isOnBdChgrSts3'	==	3	3	pas
		CAN signal 'CMD_OBC_INV_TargetPower'	==	7400 (7.4*1000)	7400	pas
		CAN signal 'CMD_OBC_INV_DC_I_Limit'	==	5	5	pas
		CAN signal 'CMD_OBC_INV_ACCurrentLimit'	==	16	16	pas
23. Check: Passed						
59.835949	1	[-] Check of expected values				
		Symbol	Op.	Reference value	Actual	Res
		System variable 'HVDC_Load::PowerMeasure'	>	1000	3009	pas





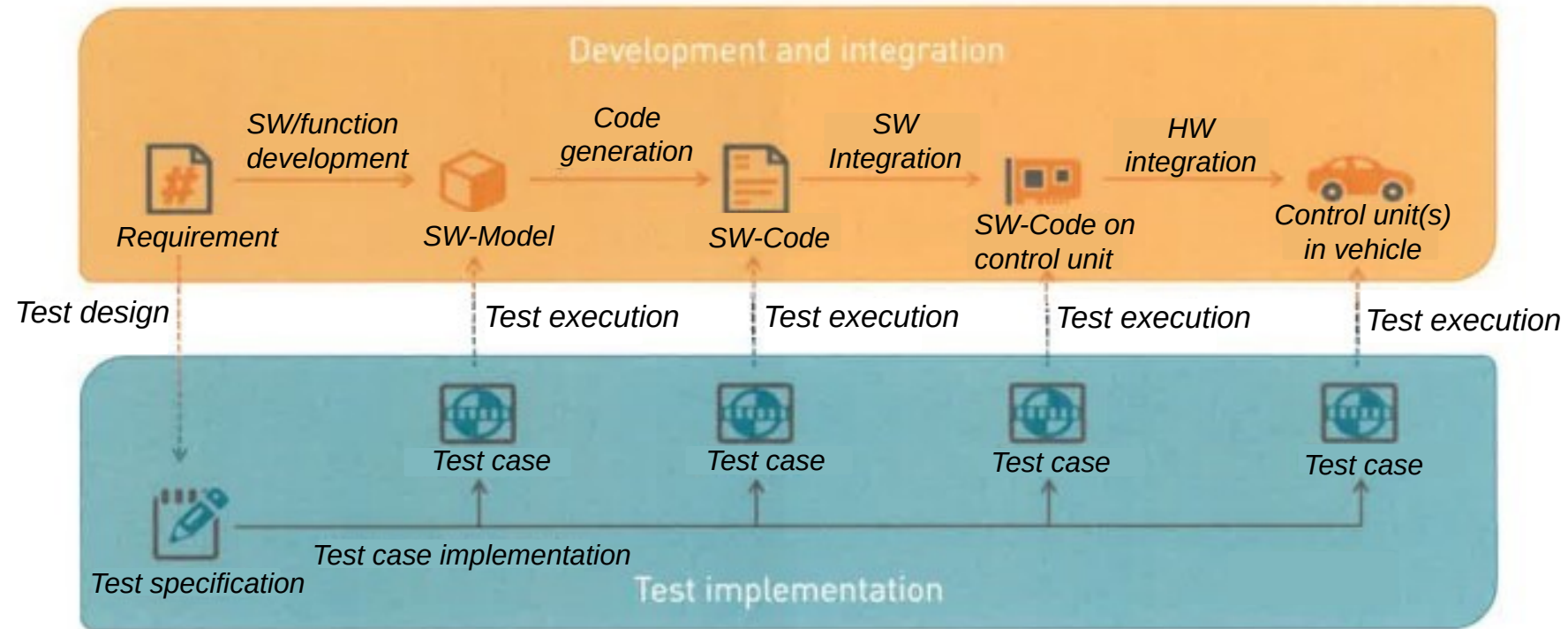
ECU-TEST is a software tool developed by **TraceTronic** for test and validation of embedded systems and used as standard tool in the development of automotive ECUs.

Main features

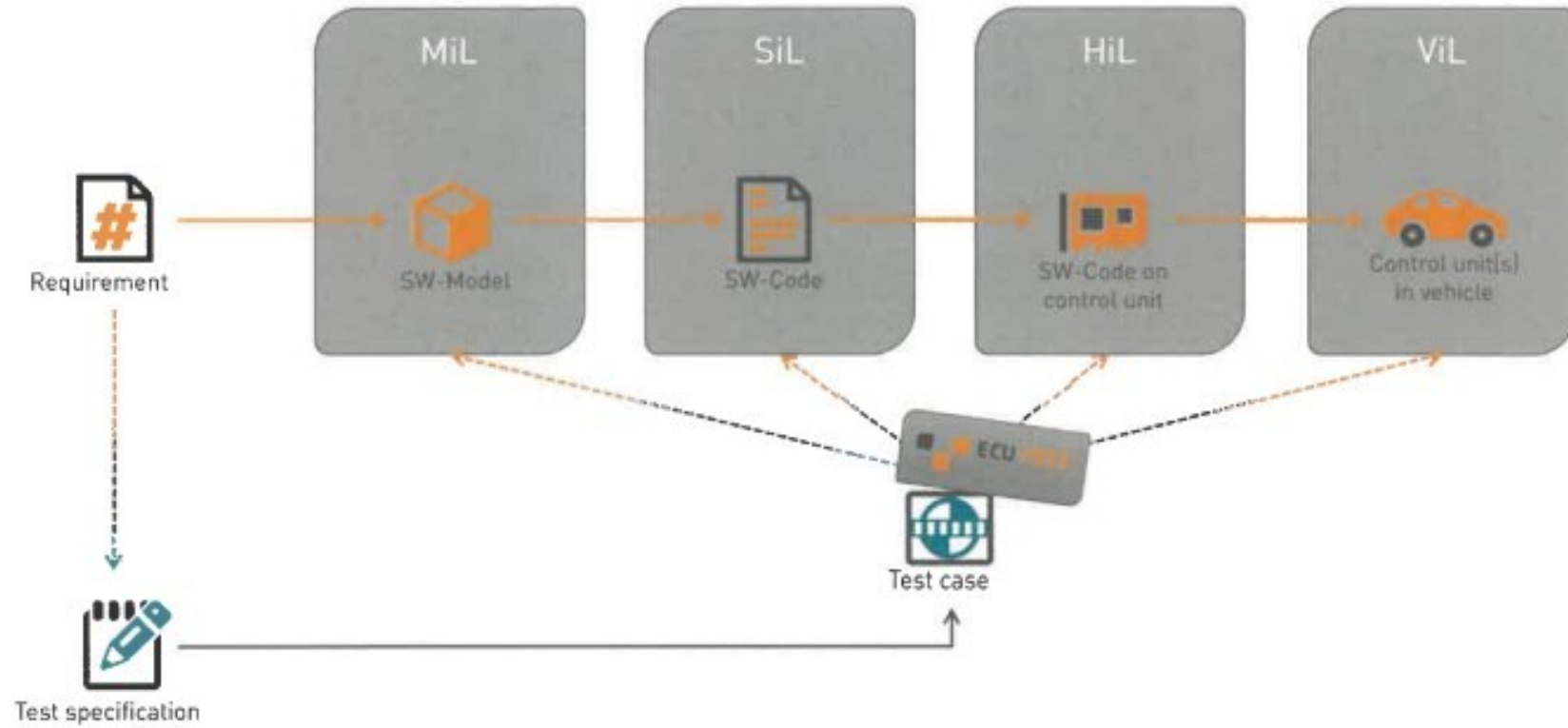
- Allows to automate the control of the whole test environment
- Supports a broad range of test tools
- Facilitate each validation step
- Is based on very user-friendly commands and interfaces
- Does not require programming skills
- Allows to use shared library



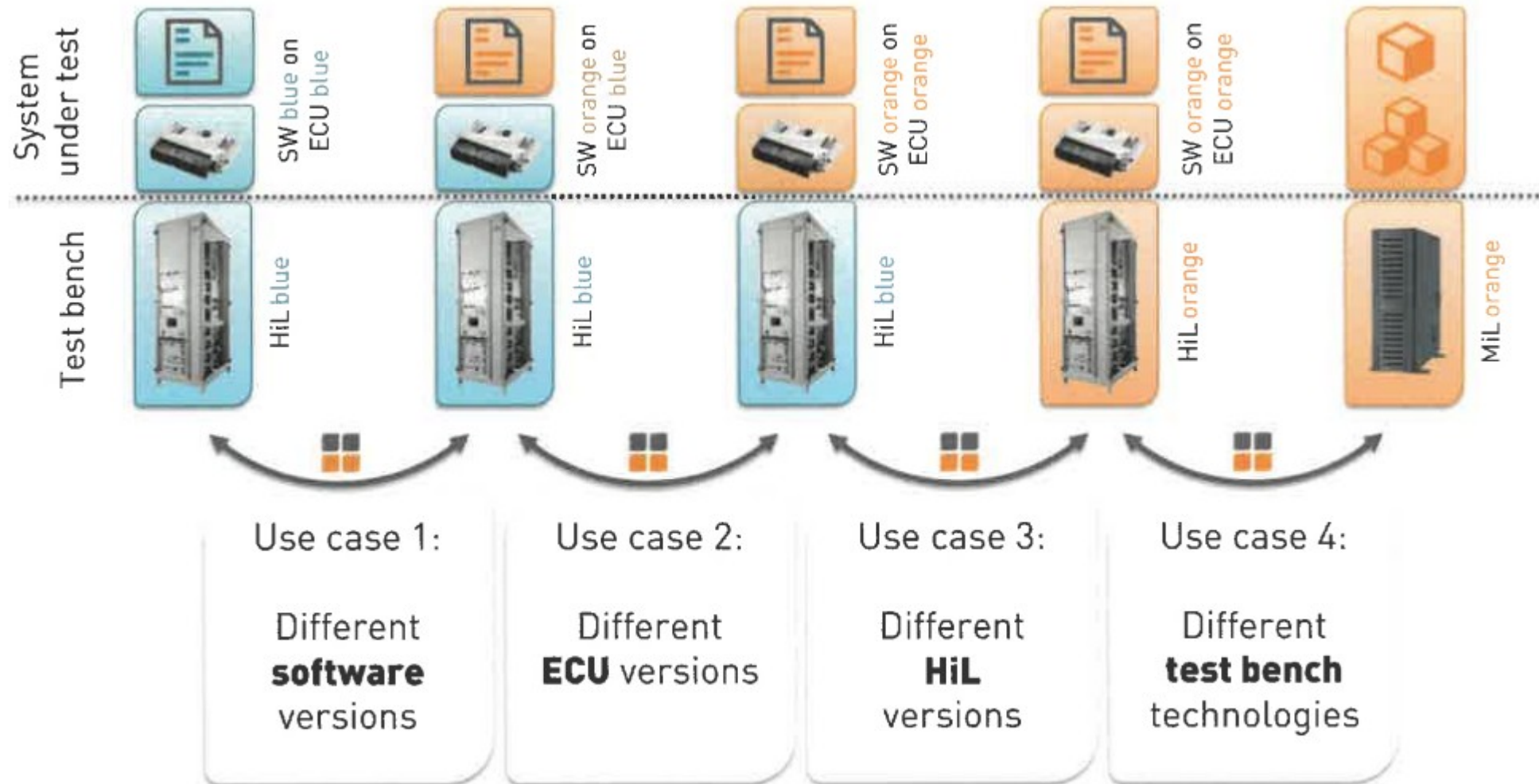
ECU-TEST – Software workflow



ECU-TEST – Test environments

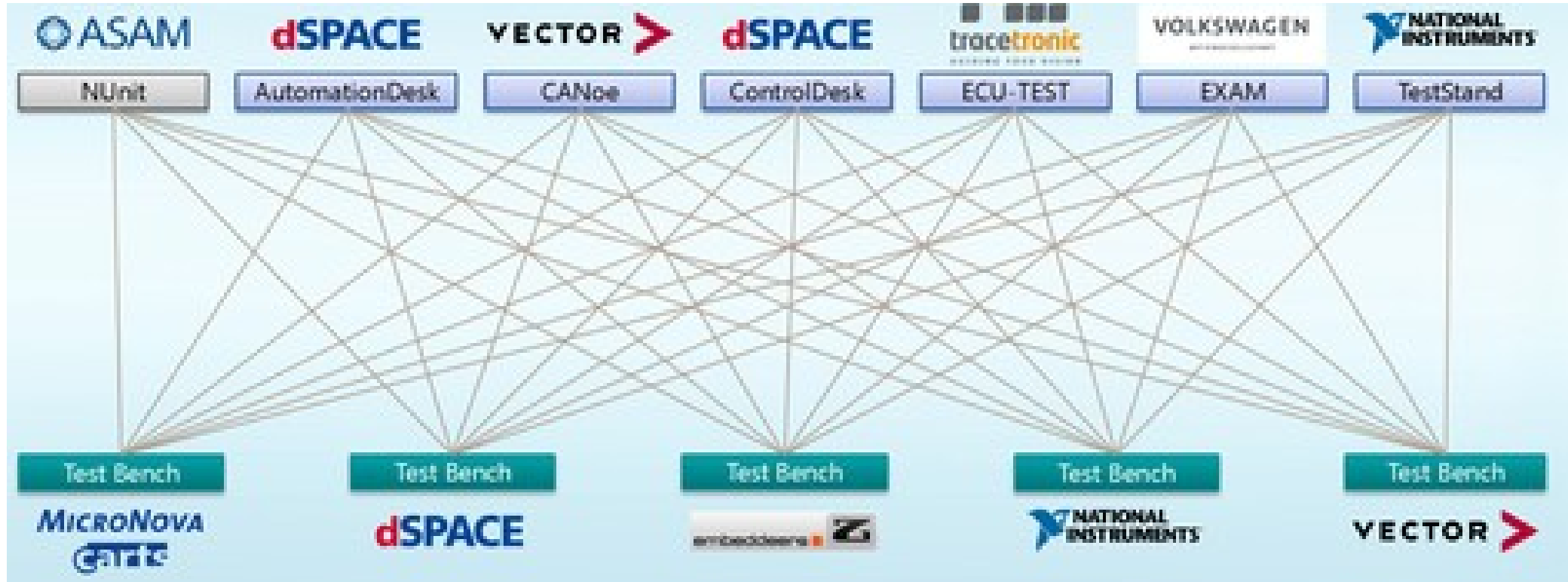


k ECU-TEST - What changes if...

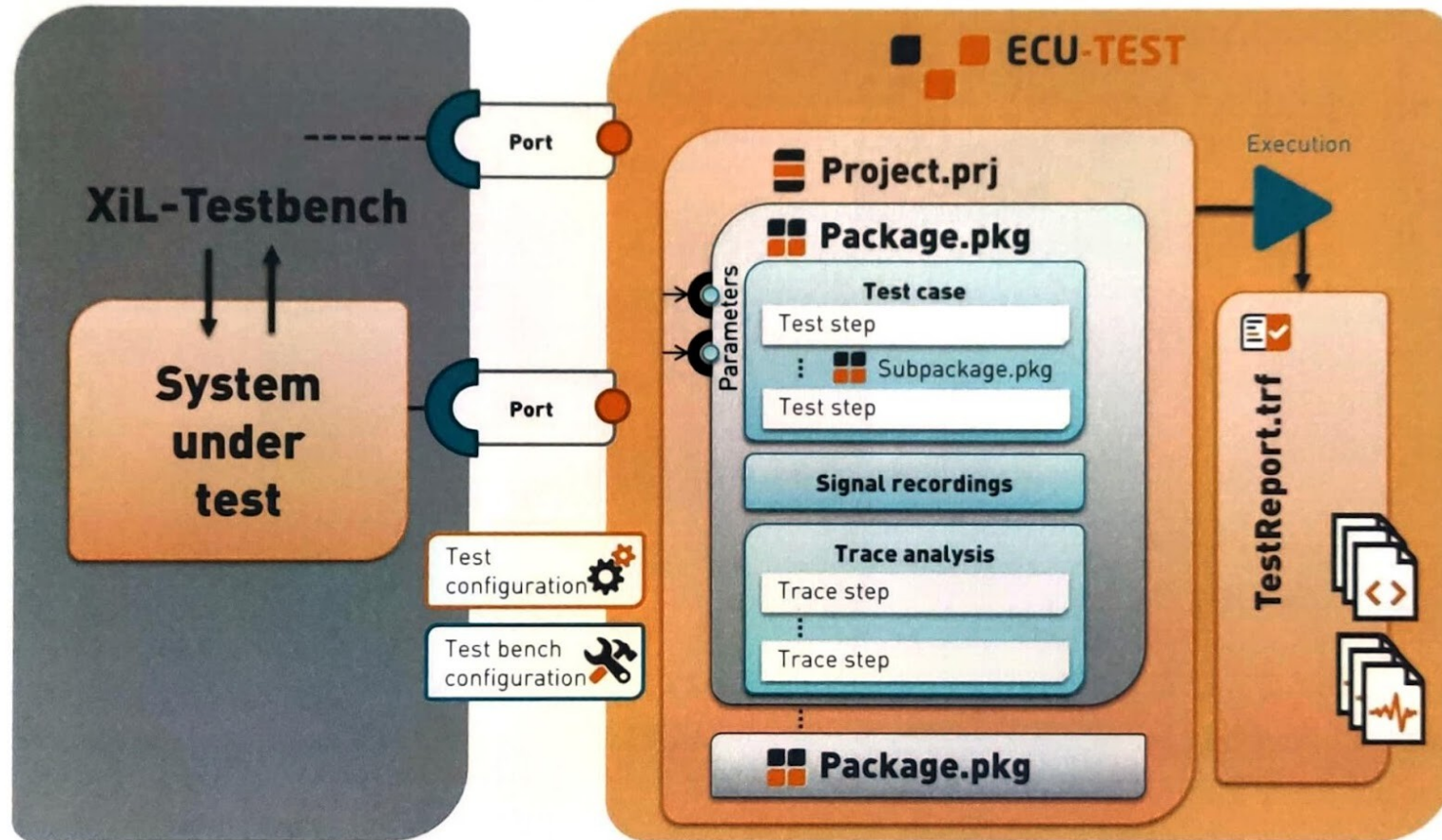




ECU-TEST - Range of tools to connect

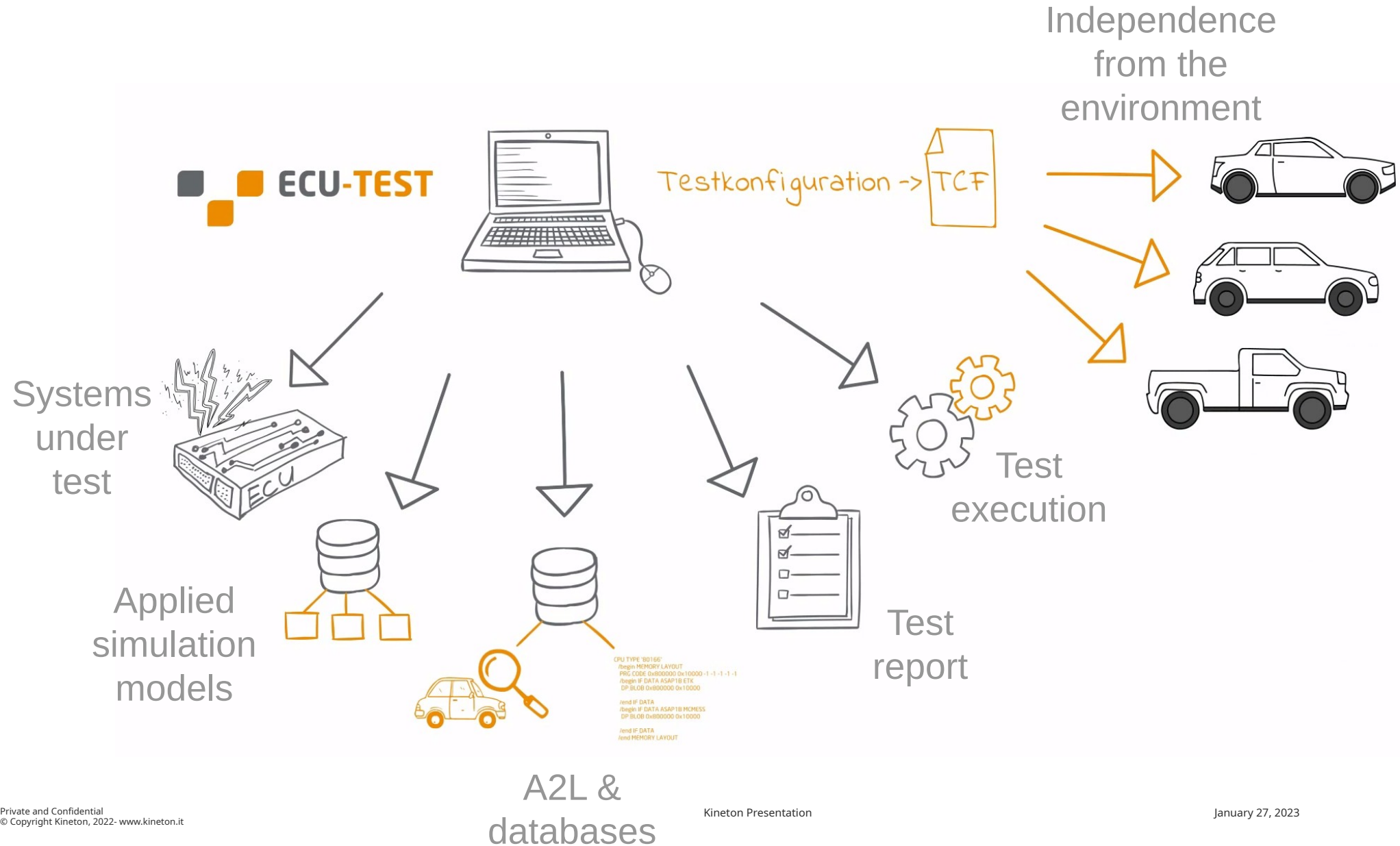


ECU-TEST - General structure



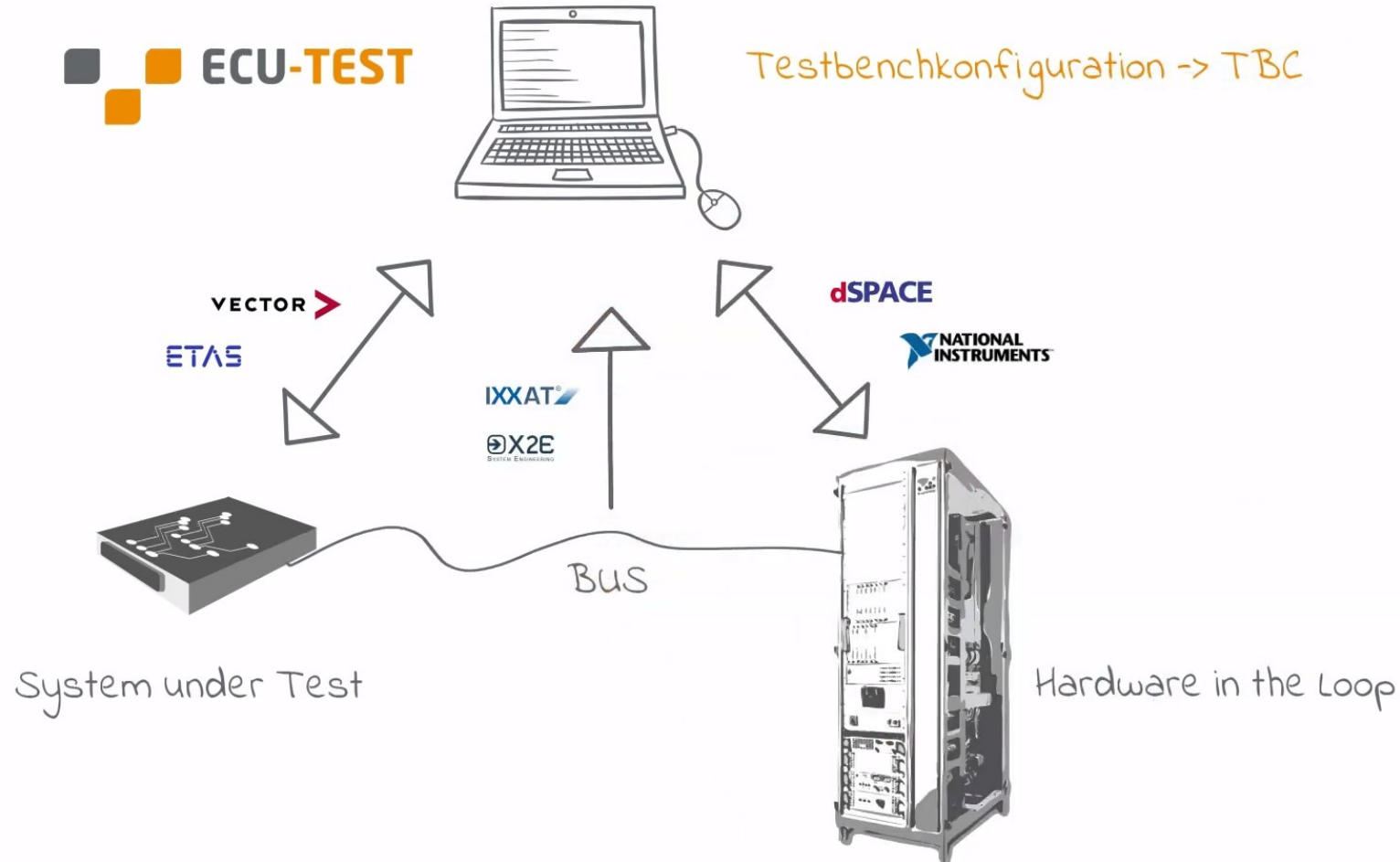


ECU-TEST - TestConfiguration - TCF





ECU-TEST - Test Bench Configuration – TBC





ECU-TEST - Test Structure

The screenshot displays the ECU-TEST software interface, specifically the 'Editor' window for a test case named 'Package 0'. The interface is divided into several panes:

- Global mapping:** A table showing mappings between model components and targets.
- Actions:** A list of available actions categorized under 'Bus access', 'Test steps', 'Model access', 'Jobs', and 'Diagnosis'. The 'Flow control' category is expanded, showing actions like Break, Continue, Exit, If-Then-Else, IfDef, Loop, React-On, Return, and Switch-Case.
- Test case Editor:** A table listing the steps of the test case.

#	Action / Name	Parameter	Expectation / Value
11	Else		
12	MODEL-Write: rBrakeFOL	PHYS(don't care)	1
13	Wait	500 ms	
14	MODEL-Write: BEPBReleased	PHYS(don't care)	1
15	Wait	500 ms	
16	MODEL-Write: BEPBReleased	PHYS(don't care)	0
17	BUS-Read: NEPBStatus	PHYS(-)	F 2000 ms 0
18	MODEL-Write: BEPBPressed	PHYS(don't care)	1
19	Wait	500 ms	
20	MODEL-Write: BEPBPressed	PHYS(don't care)	0
21	BUS-Read: NEPBStatus	PHYS(-)	F 2000 ms 3
22	BUS-Read: NEPBStatus	PHYS(-)	F 2000 ms 1
23	MODEL-Write: rBrakeFOL	PHYS(don't care)	0

Below the test case table, there is a 'Variables' section with a table:

Variables	Mapping	Name	Initial value	Current value	Description
0	☐ ☐ ☐	EPB_Current_Status	0 <Numeric>	1.0	

The status bar at the bottom indicates 'Indexing in progress...' and includes a 'Help' button.

ECU-TEST - Test Execution

Test execution: Package 0

Package progress 0:00:03

Test case Trace analysis Report

Test case

- Package 0
 - Preconditions
 - Actions
 - BUS-Read: NEPBEngaged
 - If (EPB_Current_Status == 0)
 - Then
 - Else
 - MODEL-Write: rBrakeFOL
 - Wait
 - MODEL-Write: BEPBReleased
 - Wait
 - MODEL-Write: BEPBReleased
 - BUS-Read: NEPBStatus (2 seconds remaining)
 - MODEL-Write: BEBPPressed
 - Wait
 - MODEL-Write: BEBPPressed
 - BUS-Read: NEPBStatus
 - BUS-Read: NEPBStatus
 - MODEL-Write: rBrakeFOL
 - Postconditions

Comment:

7 0 0 0 0 0

Cancel

ECU-TEST - Report Analysis

ECU-TEST package report

2023-10-13 10:14:28.655226 (UTC +1:00)

Evaluation: SUCCESS

Mapping		Target	Type	Description	Origin
BEPBPressed	Plant model/P16_Labcar_IO_model/Model Root/BEPBRelease/Value	MODEL			[Global mapping] Unbekannter MappingSpace
NEPBStatus	Torque-CAN/ABS_ESP_Modulator/EPB_CANHS_R_02//NEPBStatus	BUS	EPB status		[Global mapping] Packages\A_Test_Case\A.xam
BEPBReleased	Plant model/P16_Labcar_IO_model/Model Root/BEPBPressed/Value	MODEL			[Global mapping] Unbekannter MappingSpace
rBrakeFOL	Plant model/P16_Labcar_IO_model/Model Root/rBrakeFOL/Value	MODEL			[Global mapping] Unbekannter MappingSpace
NEPBEngaged	Torque-CAN/ABS_ESP_Modulator/EPB_CANHS_R_02//NEPBEngaged	BUS	Current status of the EPB		[Global mapping] Packages\A_Test_Case\A.xam

Test case		Action	Name	Value	Expected value	Comment
#	Time [s]					
1	0.001	Preconditions				
2	0.001	Actions				
3	0.002	BUS-Read	NEPBEngaged	1.0		
4	0.023	If (EPB_Current_Status == 0)	If-Then-Else	False		
11	0.023	Else				
12	0.023	MODEL-Write	rBrakeFOL	1		
13	0.086	UTILITY	Wait	500 ms		
14	0.587	MODEL-Write	BEPBReleased	1		
15	0.623	UTILITY	Wait	500 ms		
16	1.125	MODEL-Write	BEPBReleased	0		
17	1.179	BUS-Read	NEPBStatus	0.0 (Wait until true: 70 trials [1013.7529999999999 ms])	0	
18	2.214	MODEL-Write	BEPBPressed	1		
19	2.264	UTILITY	Wait	500 ms		
20	2.765	MODEL-Write	BEPBPressed	0		
21	2.826	BUS-Read	NEPBStatus	3.0 (Wait until true: 1 trials [0.0 ms])	3	
22	2.842	BUS-Read	NEPBStatus	1.0 (Wait until true: 83 trials [1101.114 ms])	1	
23	3.961	MODEL-Write	rBrakeFOL	0		
24	4.016	Postconditions				



Comparison

	CAPL	vTESTstudio	ECU-TEST
Pre-settings needed	CANoe project settings	CANoe project settings	All the linked tools + ECU-TEST settings
Test implementation procedure	C code writing	Drag & drop pre-defined functions	Drag & drop pre-defined functions
Programming skills needed	Directly proportional to test complexity	Limited	No one
Auto-generated test report	Customisable in each test code	Rich of details	Simple and intuitive
Robustness against changes	Low	Low	Proportional to proper feature usage



CONTACTS



Massimiliano Schettino
massimiliano.schettino@kineton.it